

Matplotlib Cheat Sheet

A practical quick reference for beginners & intermediate users
Ordered from the most commonly used functions to the least

EVERY EXAMPLE IN THIS GUIDE USES THIS SETUP

```
import matplotlib.pyplot as plt
import pandas as pd

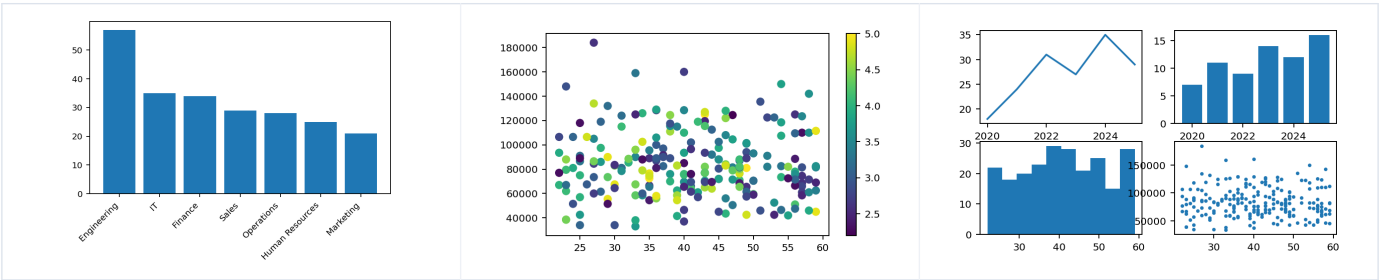
df = pd.read_csv("HR_ListOfEmployees.csv")

# summaries reused throughout this guide
dept_counts = df["Department"].value_counts()
dept_pay = df.groupby("Department")["Salary"].mean()
years = [2020, 2021, 2022, 2023, 2024, 2025]
hires = [18, 24, 31, 27, 35, 29]
leavers = [7, 11, 9, 14, 12, 16]
```

FOUR IDEAS THAT EXPLAIN MOST OF MATPLOTLIB

plt	The quick way: plt.plot(), plt.bar(), then plt.show().
Figure / Axes	fig is the whole page; ax is one chart drawn on it.
fig, ax = subplots()	The grown-up way - needed for multiple charts.
plt.show()	Displays the chart; plt.savefig() writes it to a file.

WHAT THE SAMPLE OUTPUTS LOOK LIKE



Every one of the 113 charts in this guide is a real image rendered by running the code beside it.

Blue bar = code you type **Green bar** = the chart it draws **125 examples · 15 topics**

Companion to the Pandas, NumPy & Scikit-Learn cheat sheets — all built on the same HR dataset.

Table of Contents

01	Setup & Your First Plot Import the library and draw something.	9 functions	3
02	Line Charts Show how a value changes over time.	9 functions	4
03	Bar Charts Compare values across categories.	10 functions	6
04	Histograms See how one column is spread out.	8 functions	8
05	Scatter Plots Check if two columns are related.	7 functions	9
06	Pie & Donut Charts Show parts of a whole.	6 functions	10
07	Titles, Labels & Legends Tell the reader what they are seeing.	9 functions	11
08	Colours & Styles Make charts match your report.	9 functions	13
09	Markers & Line Styles Fine-tune how lines look.	8 functions	14
10	Axis Limits & Scales Control what range is shown.	8 functions	15
11	Grids & Ticks Make values easier to read off.	9 functions	16
12	Multiple Charts (Subplots) Put several charts on one figure.	8 functions	18
13	Annotations & Text Point out the important bits.	8 functions	20
14	Plotting Straight from Pandas The fastest way to chart a DataFrame.	9 functions	21
15	Saving & Exporting Get the chart out of Python.	8 functions	23

1. Setup & Your First Plot

9 functions

Import the library and draw something.

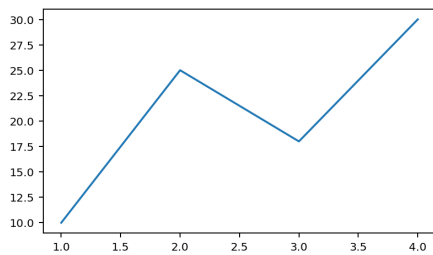
1.1 `import matplotlib.pyplot as plt` # Standard way to import Matplotlib

SAMPLE OUTPUT

plt is ready to use

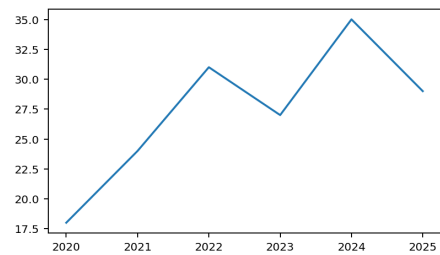
1.2 # Draws the simplest line chart
`plt.plot([1, 2, 3, 4], [10, 25, 18, 30])`

CHART



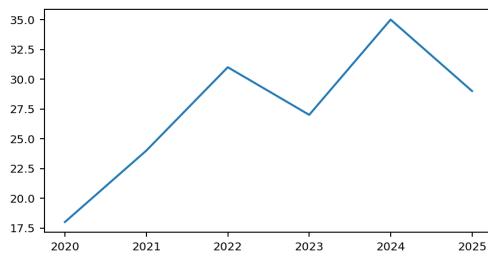
1.3 # Displays the chart on screen
`plt.plot(years, hires)`
`plt.show()`

CHART



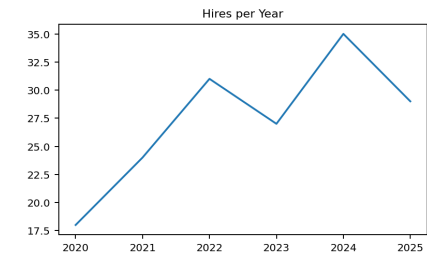
1.4 # Sets the chart size in inches
`plt.figure(figsize=(6, 3))`
`plt.plot(years, hires)`

CHART



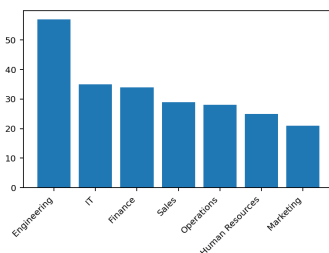
1.5 # Adds a title above the chart
`plt.plot(years, hires)`
`plt.title("Hires per Year")`

CHART



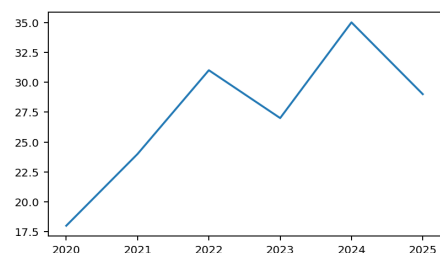
1.6 # Your first chart from real data
`plt.bar(dept_counts.index, dept_counts.values)`
`plt.xticks(rotation=45, ha="right")`

CHART



1.7 # The modern way: figure and axes
`fig, ax = plt.subplots()`
`ax.plot(years, hires)`

CHART



1.8 `plt.savefig("hires.png", dpi=150)` # Saves the chart as an image file

SAMPLE OUTPUT

1

1.9 `plt.close()` # Closes the figure to free memory

SAMPLE OUTPUT

0

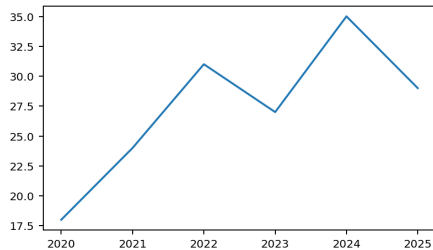
2. Line Charts

9 functions

Show how a value changes over time.

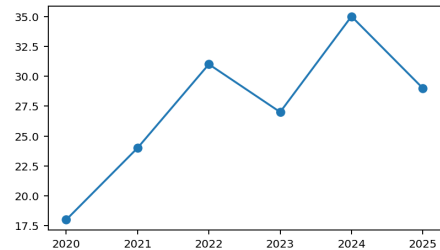
2.1 `plt.plot(years, hires)` # Basic line chart

CHART



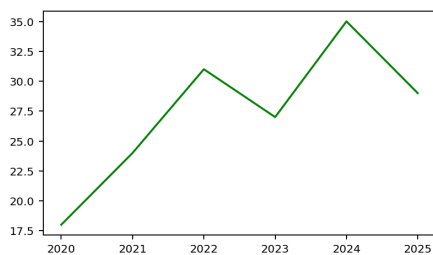
2.2 `# Adds a dot at each data point`
`plt.plot(years, hires, marker="o")`

CHART



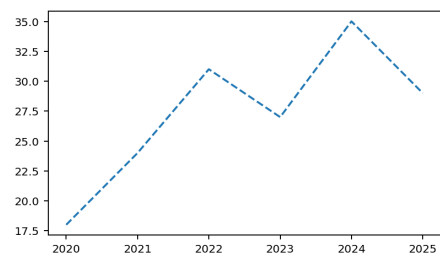
2.3 `# Changes the line colour`
`plt.plot(years, hires, color="green")`

CHART



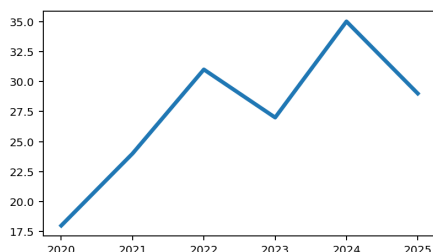
2.4 `# Makes the line dashed`
`plt.plot(years, hires, linestyle="--")`

CHART



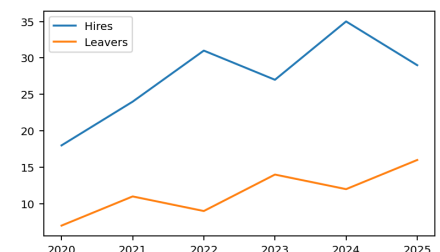
2.5 `# Makes the line thicker`
`plt.plot(years, hires, linewidth=3)`

CHART



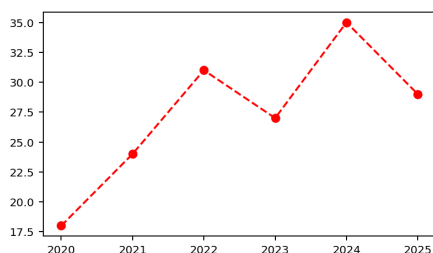
2.6 `# Two lines on one chart`
`plt.plot(years, hires, label="Hires")`
`plt.plot(years, leavers, label="Leavers")`
`plt.legend()`

CHART



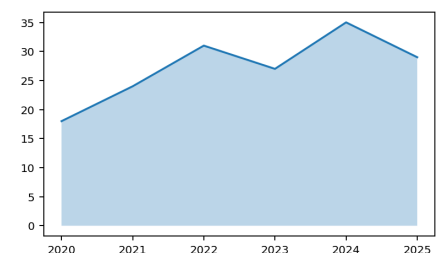
2.7 `# Shortcut for marker, line and colour`
`plt.plot(years, hires, "o--r")`

CHART



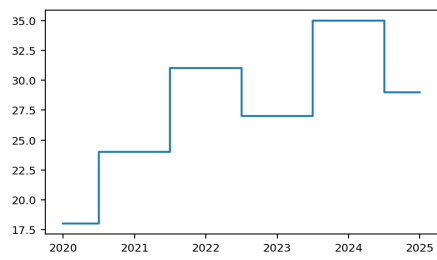
2.8 `# Shades the area under the line`
`plt.fill_between(years, hires, alpha=0.3)`
`plt.plot(years, hires)`

CHART



2.9 # Draws a stepped line
`plt.step(years, hires, where="mid")`

CHART



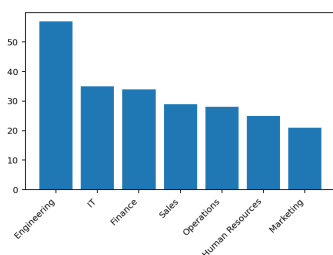
3. Bar Charts

10 functions

Compare values across categories.

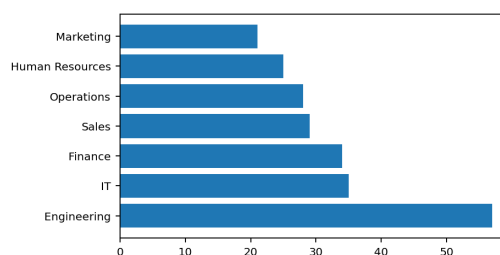
```
3.1 # Vertical bars for each category
plt.bar(dept_counts.index, dept_counts.values)
plt.xticks(rotation=45, ha="right")
```

CHART



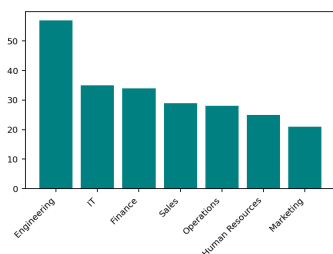
```
3.2 # Horizontal bars, better for long names
plt.barh(dept_counts.index, dept_counts.values)
```

CHART



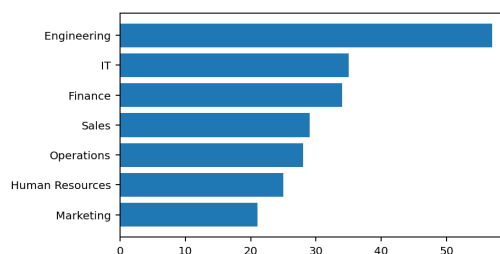
```
3.3 # Sets one colour for all bars
plt.bar(dept_counts.index, dept_counts.values, color="teal")
plt.xticks(rotation=45, ha="right")
```

CHART



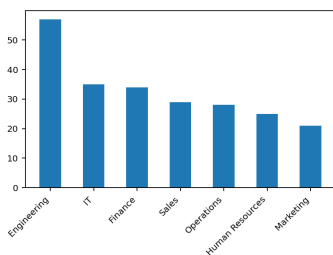
```
3.4 # Sorts bars from smallest to largest
order = dept_counts.sort_values()
plt.barh(order.index, order.values)
```

CHART



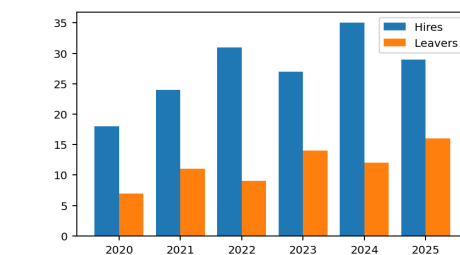
```
3.5 # Makes the bars narrower
plt.bar(dept_counts.index, dept_counts.values, width=0.5)
plt.xticks(rotation=45, ha="right")
```

CHART



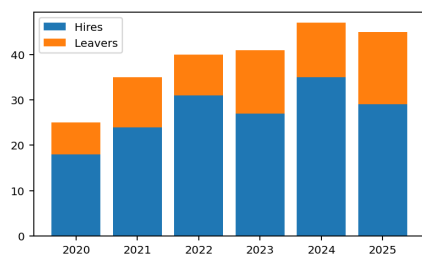
```
3.6 # Grouped bars side by side
x = np.arange(len(years))
plt.bar(x - 0.2, hires, 0.4, label="Hires")
plt.bar(x + 0.2, leavers, 0.4, label="Leavers")
plt.xticks(x, years)
plt.legend()
```

CHART



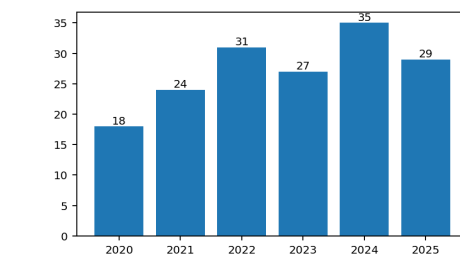
```
3.7 # Stacked bars on top of each other
plt.bar(years, hires, label="Hires")
plt.bar(years, leavers, bottom=hires, label="Leavers")
plt.legend()
```

CHART



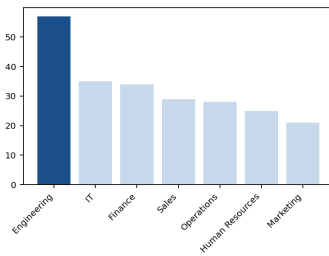
```
3.8 # Writes the value on each bar
bars = plt.bar(years, hires)
plt.bar_label(bars)
```

CHART



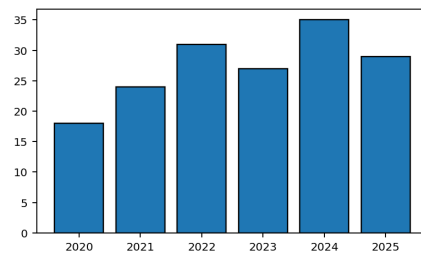
```
3.9 # Highlights bars above a cut-off
plt.bar(dept_counts.index, dept_counts.values,
        color=["#1a4f8a" if v > 35 else "#c7d8ea"
              for v in dept_counts.values])
plt.xticks(rotation=45, ha="right")
```

CHART



```
3.10 # Adds an outline to each bar
plt.bar(years, hires, edgecolor="black")
```

CHART



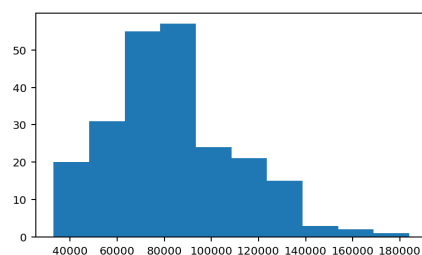
4. Histograms

8 functions

See how one column is spread out.

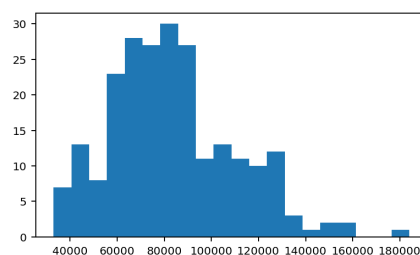
4.1 `plt.hist(df["Salary"])` # Shows the shape of your data

CHART



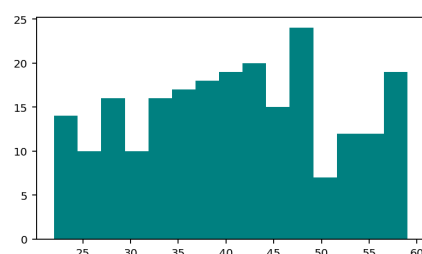
4.2 # More bins means more detail
`plt.hist(df["Salary"], bins=20)`

CHART



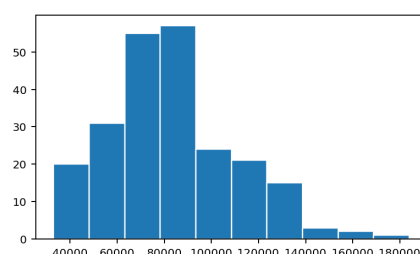
4.3 # Sets the bar colour
`plt.hist(df["Age"], bins=15, color="teal")`

CHART



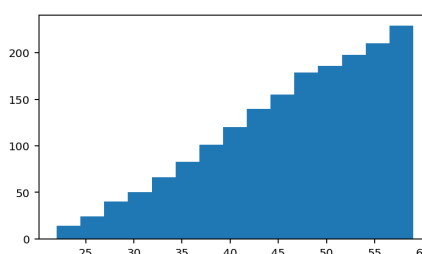
4.4 # Separates the bars clearly
`plt.hist(df["Salary"], edgecolor="white")`

CHART



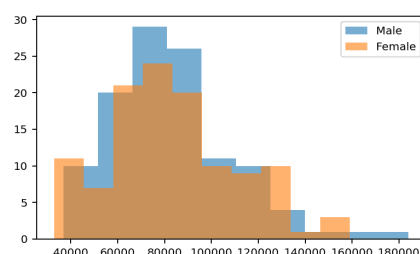
4.5 # Adds counts up as it goes
`plt.hist(df["Age"], bins=15, cumulative=True)`

CHART



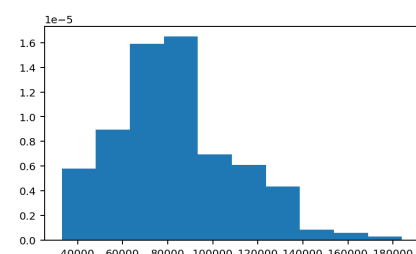
4.6 # Compares two groups at once
`plt.hist(df[df["Gender"] == "Male"]["Salary"], alpha=0.6, label="Male")`
`plt.hist(df[df["Gender"] == "Female"]["Salary"], alpha=0.6, label="Female")`
`plt.legend()`

CHART



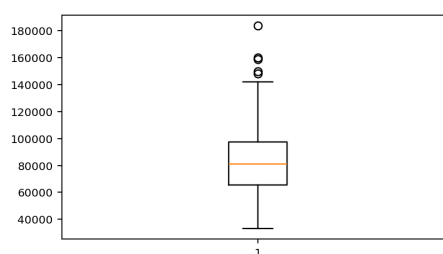
4.7 # Shows proportion instead of count
`plt.hist(df["Salary"], density=True)`

CHART



4.8 # Box plot shows median and outliers
`plt.boxplot(df["Salary"].dropna())`

CHART



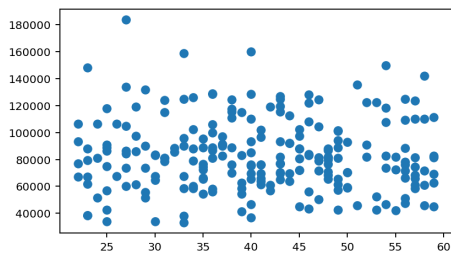
5. Scatter Plots

7 functions

Check if two columns are related.

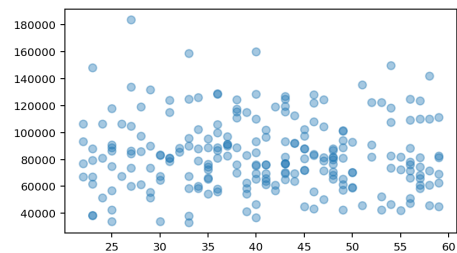
5.1 # One dot per employee
`plt.scatter(df["Age"], df["Salary"])`

CHART



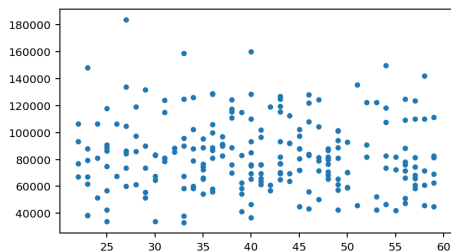
5.2 # See-through dots show crowding
`plt.scatter(df["Age"], df["Salary"], alpha=0.4)`

CHART



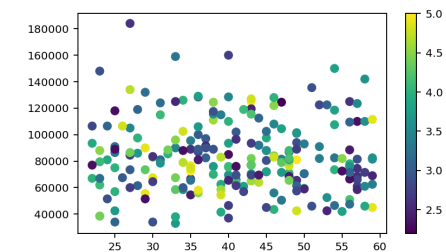
5.3 # Makes the dots smaller
`plt.scatter(df["Age"], df["Salary"], s=10)`

CHART



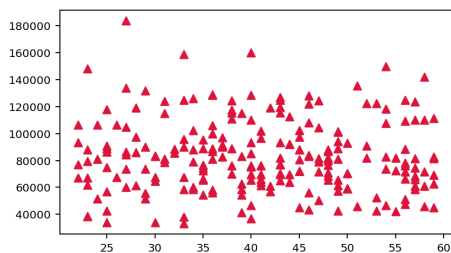
5.4 # Colours dots by a third column
`plt.scatter(df["Age"], df["Salary"],
 c=df["Performance_Rating"], cmap="viridis")`
`plt.colorbar()`

CHART



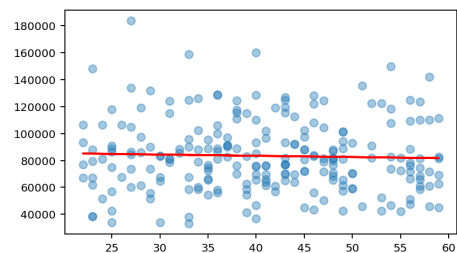
5.5 # Changes the dot shape and colour
`plt.scatter(df["Age"], df["Salary"], c="crimson", marker="^")`

CHART



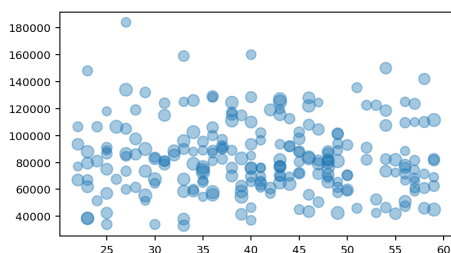
5.6 # Adds a straight trend line
`plt.scatter(df["Age"], df["Salary"], alpha=0.4)`
`m, b = np.polyfit(df["Age"], df["Salary"], 1)`
`plt.plot(df["Age"], m * df["Age"] + b, "r-")`

CHART



5.7 # Dot size shows another value
`plt.scatter(df["Age"], df["Salary"],
 s=df["Performance_Rating"] * 20, alpha=0.4)`

CHART



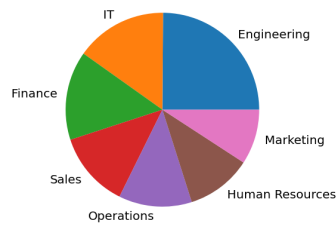
6. Pie & Donut Charts

6 functions

Show parts of a whole.

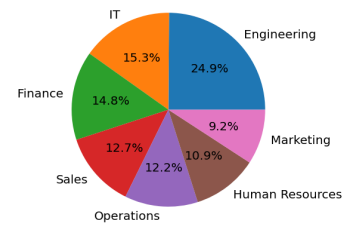
6.1 # Slices sized by each category
`plt.pie(dept_counts.values, labels=dept_counts.index)`

CHART



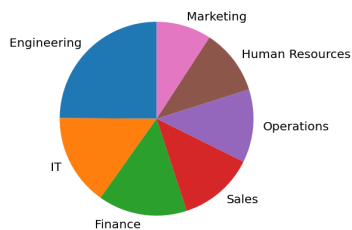
6.2 # Writes the percentage on each slice
`plt.pie(dept_counts.values, labels=dept_counts.index, autopct="%1.1f%%")`

CHART



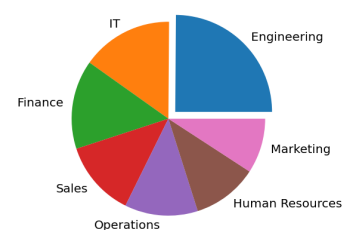
6.3 # Rotates where the first slice starts
`plt.pie(dept_counts.values, labels=dept_counts.index, startangle=90)`

CHART



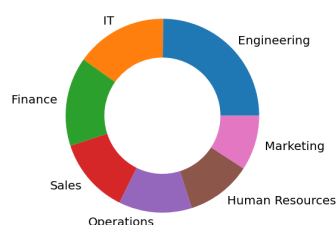
6.4 # Pulls one slice out for emphasis
`explode = [0.1] + [0] * 5`
`plt.pie(dept_counts.values, labels=dept_counts.index, explode=explode)`

CHART



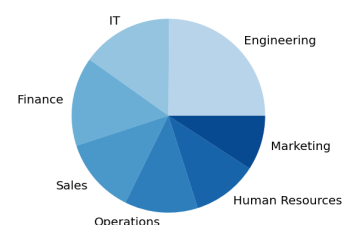
6.5 # Turns the pie into a donut
`plt.pie(dept_counts.values, labels=dept_counts.index, wedgeprops={'width': 0.4})`

CHART



6.6 # Uses a colour range for slices
`plt.pie(dept_counts.values, labels=dept_counts.index, colors=plt.cm.Blues(np.linspace(0.3, 0.9, 7)))`

CHART



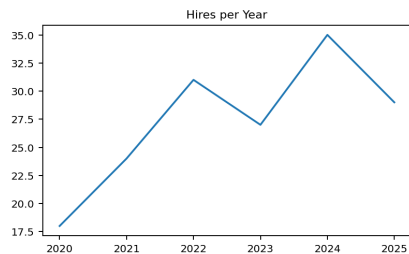
7. Titles, Labels & Legends

9 functions

Tell the reader what they are seeing.

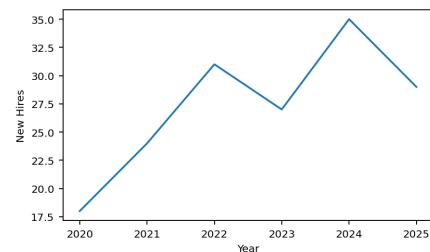
```
7.1 # Adds the main chart title
plt.plot(years, hires)
plt.title("Hires per Year")
```

CHART



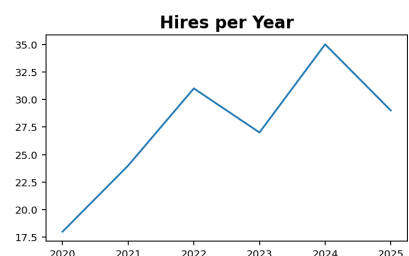
```
7.2 # Names the bottom and side axes
plt.plot(years, hires)
plt.xlabel("Year")
plt.ylabel("New Hires")
```

CHART



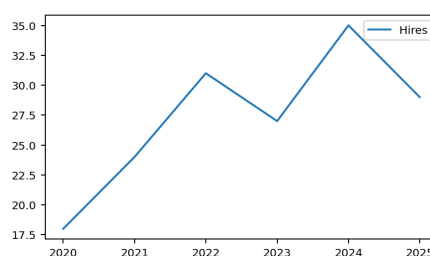
```
7.3 # Makes the title bigger and bold
plt.plot(years, hires)
plt.title("Hires per Year", fontsize=14, fontweight="bold")
```

CHART



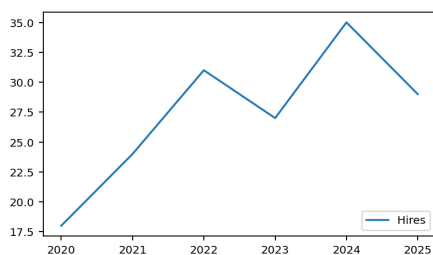
```
7.4 # Adds a key explaining each line
plt.plot(years, hires, label="Hires")
plt.legend()
```

CHART



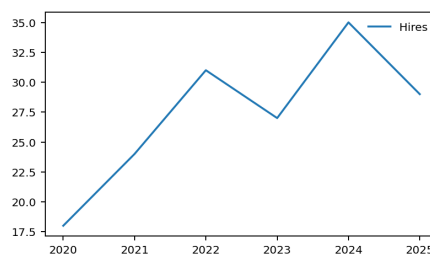
```
7.5 # Moves the legend to a corner
plt.plot(years, hires, label="Hires")
plt.legend(loc="lower right")
```

CHART



```
7.6 # Removes the box around the legend
plt.plot(years, hires, label="Hires")
plt.legend(frameon=False)
```

CHART



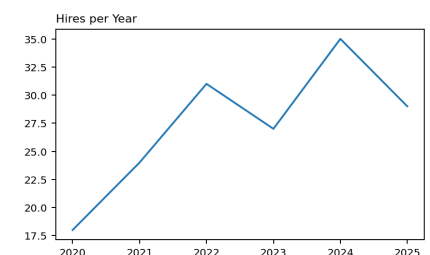
```
7.7 # Adds a heading above the title
plt.plot(years, hires)
plt.suptitle("HR Report")
plt.title("Hires per Year")
```

CHART

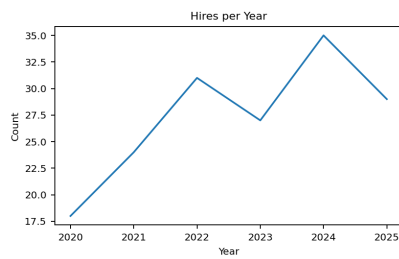


```
7.8 # Aligns the title to the left
plt.plot(years, hires)
plt.title("Hires per Year", loc="left")
```

CHART



7.9 # Sets everything in one line
fig, ax = plt.subplots()
ax.plot(years, hires)
ax.set(title="Hires per Year", xlabel="Year", ylabel="Count")

CHART

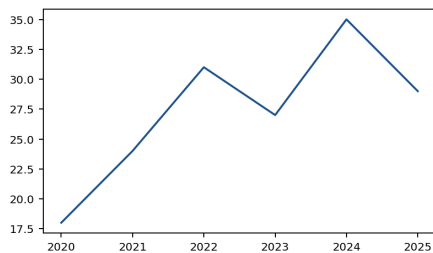
8. Colours & Styles

9 functions

Make charts match your report.

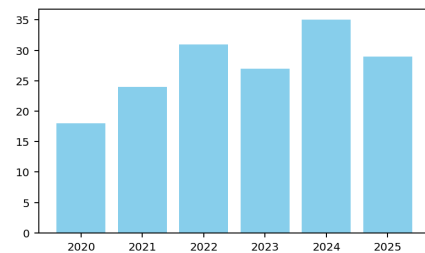
8.1 `# Uses an exact colour code`
`plt.plot(years, hires, color="#1a4f8a")`

CHART



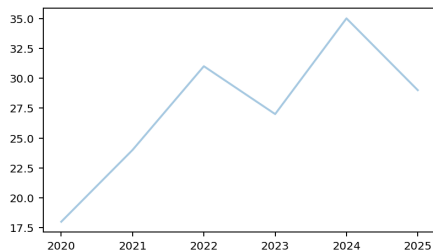
8.2 `# Uses a named colour`
`plt.bar(years, hires, color="skyblue")`

CHART



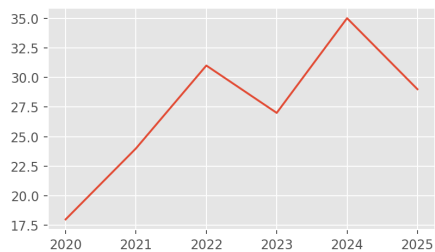
8.3 `# Makes the line see-through`
`plt.plot(years, hires, alpha=0.4)`

CHART



8.4 `# Applies a ready-made theme`
`plt.style.use("ggplot")`
`plt.plot(years, hires)`

CHART



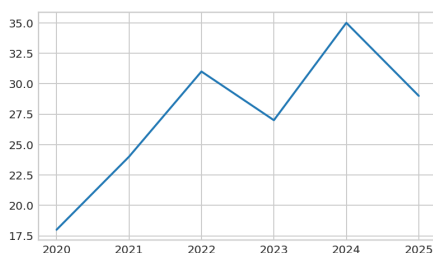
8.5 `plt.style.available[:6]` `# Lists the themes you can use`

SAMPLE OUTPUT

```
['Solarize_Light2', '_classic_test_patch', '_mpl-gallery', '_mpl-gallery-nogrid', 'bmh', 'classic']
```

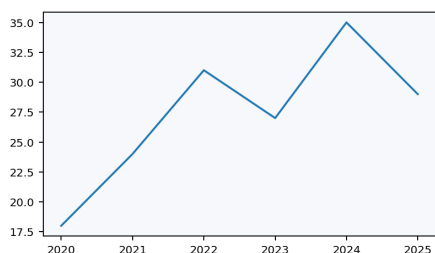
8.6 `# A clean theme good for reports`
`plt.style.use("seaborn-v0_8-whitegrid")`
`plt.plot(years, hires)`

CHART



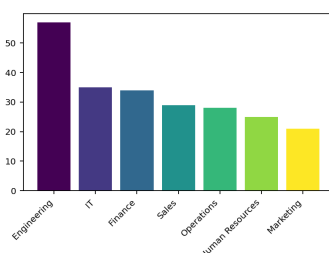
8.7 `# Changes the background colour`
`plt.plot(years, hires)`
`plt.gca().set_facecolor("#f6f8fb")`

CHART



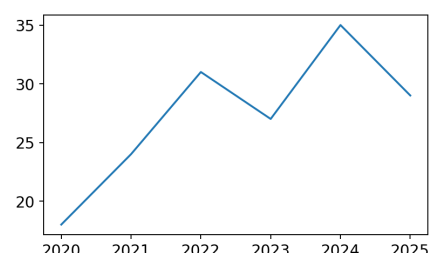
8.8 `# Gives every bar a different colour`
`colors = plt.cm.viridis(np.linspace(0, 1, 7))`
`plt.bar(dept_counts.index, dept_counts.values, color=colors)`
`plt.xticks(rotation=45, ha="right")`

CHART



8.9 `# Sets a default for every chart`
`plt.rcParams["font.size"] = 12`
`plt.plot(years, hires)`

CHART



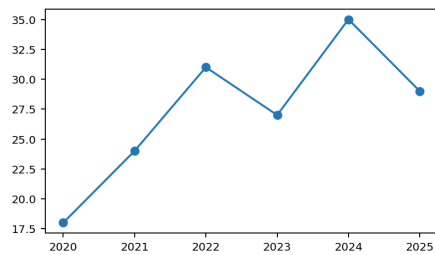
9. Markers & Line Styles

8 functions

Fine-tune how lines look.

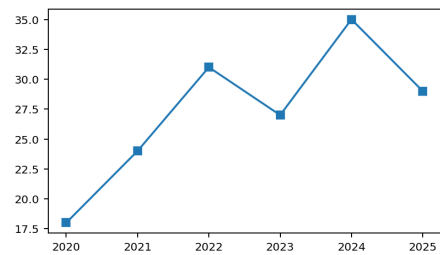
9.1 `plt.plot(years, hires, marker="o")` # Circle markers

CHART



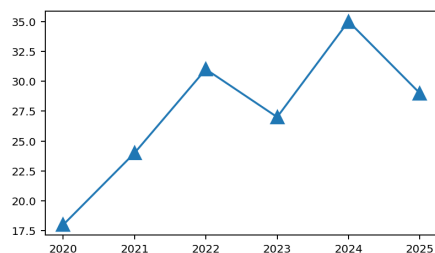
9.2 `plt.plot(years, hires, marker="s")` # Square markers

CHART



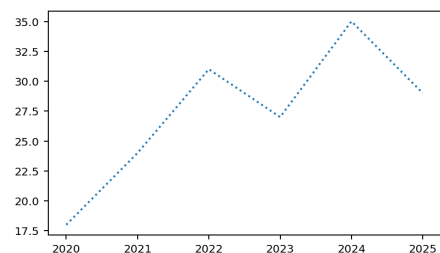
9.3 # Bigger triangle markers
`plt.plot(years, hires, marker="^", markersize=10)`

CHART



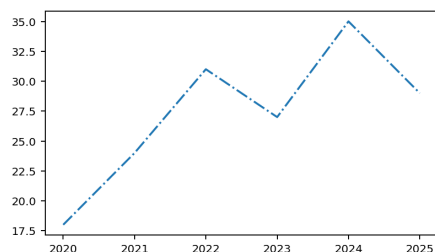
9.4 `plt.plot(years, hires, linestyle=":")` # Dotted line

CHART



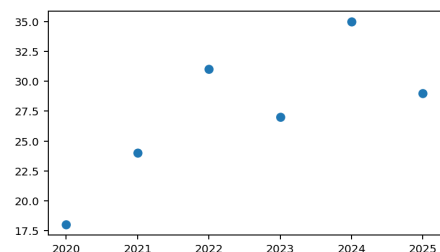
9.5 # Dash-dot line
`plt.plot(years, hires, linestyle="-.")`

CHART



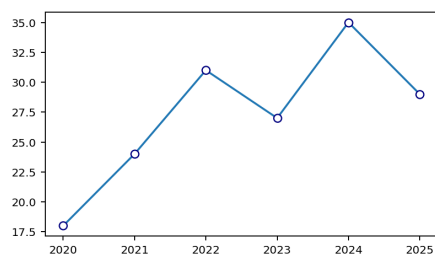
9.6 # Markers only, no line
`plt.plot(years, hires, linestyle="none", marker="o")`

CHART



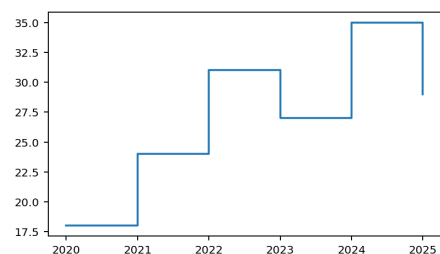
9.7 # Hollow markers with an outline
`plt.plot(years, hires, marker="o", markerfacecolor="white", markeredgcolor="navy")`

CHART



9.8 # Line that jumps in steps
`plt.plot(years, hires, drawstyle="steps-post")`

CHART



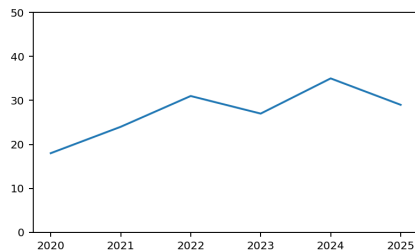
10. Axis Limits & Scales

8 functions

Control what range is shown.

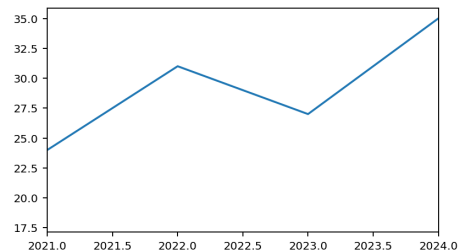
10.1 # Sets the vertical range
`plt.plot(years, hires)`
`plt.ylim(0, 50)`

CHART



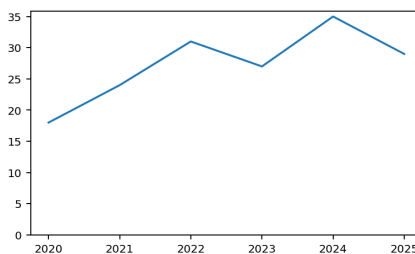
10.2 # Sets the horizontal range
`plt.plot(years, hires)`
`plt.xlim(2021, 2024)`

CHART



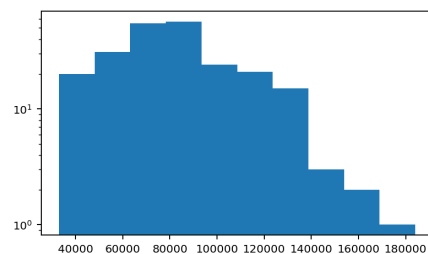
10.3 # Starts the axis at zero, always safer
`plt.plot(years, hires)`
`plt.ylim(bottom=0)`

CHART



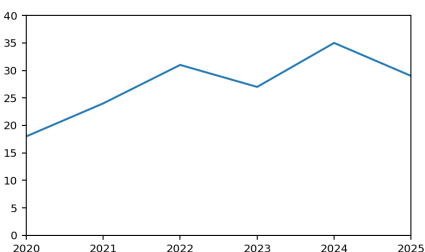
10.4 # Log scale for very uneven data
`plt.hist(df["Salary"])`
`plt.yscale("log")`

CHART



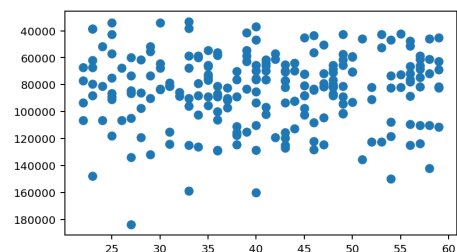
10.5 # Sets both ranges at once
`plt.plot(years, hires)`
`plt.axis([2020, 2025, 0, 40])`

CHART



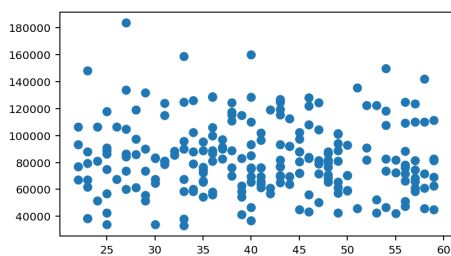
10.6 # Flips the axis upside down
`plt.scatter(df["Age"], df["Salary"])`
`plt.gca().invert_yaxis()`

CHART



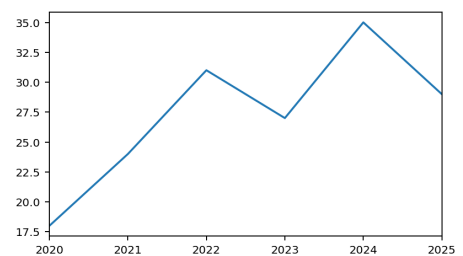
10.7 # Controls the width-to-height ratio
`plt.scatter(df["Age"], df["Salary"])`
`plt.gca().set_aspect("auto")`

CHART



10.8 # Removes padding at the edges
`plt.plot(years, hires)`
`plt.margins(x=0)`

CHART



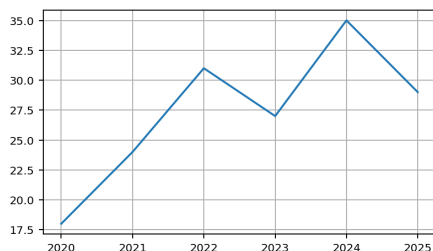
11. Grids & Ticks

9 functions

Make values easier to read off.

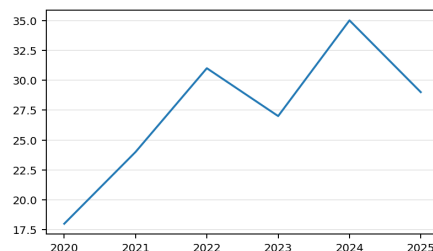
```
11.1 # Adds grid lines
plt.plot(years, hires)
plt.grid(True)
```

CHART



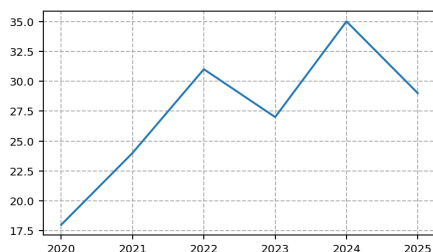
```
11.2 # Faint grid lines on one axis only
plt.plot(years, hires)
plt.grid(axis="y", alpha=0.3)
```

CHART



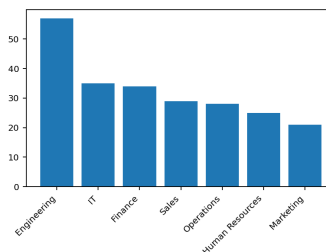
```
11.3 # Dashed grid lines
plt.plot(years, hires)
plt.grid(True, linestyle="--")
```

CHART



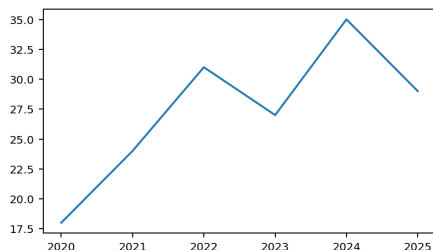
```
11.4 # Rotates labels so they fit
plt.bar(dept_counts.index, dept_counts.values)
plt.xticks(rotation=45, ha="right")
```

CHART



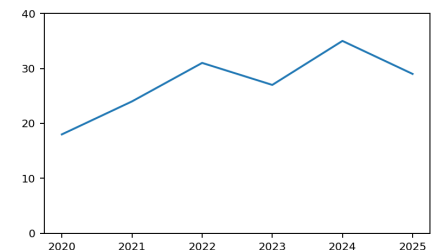
```
11.5 # Shows every value on the axis
plt.plot(years, hires)
plt.xticks(years)
```

CHART



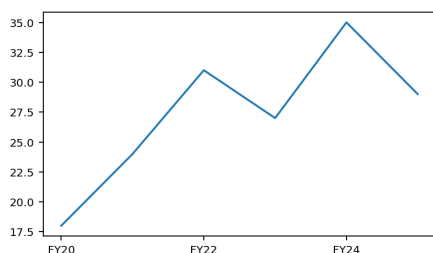
```
11.6 # Chooses exactly which ticks appear
plt.plot(years, hires)
plt.yticks([0, 10, 20, 30, 40])
```

CHART



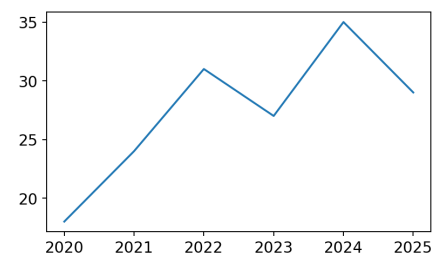
```
11.7 # Replaces tick labels with your own
plt.plot(years, hires)
plt.xticks([2020, 2022, 2024], ["FY20", "FY22", "FY24"])
```

CHART



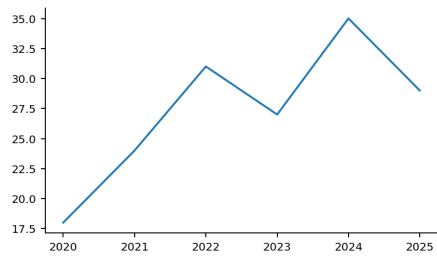
```
11.8 # Makes the tick numbers bigger
plt.plot(years, hires)
plt.tick_params(labelsize=12)
```

CHART




```
11.9 # Removes the top and right borders
fig, ax = plt.subplots()
ax.plot(years, hires)
ax.spines[["top", "right"]].set_visible(False)
```

CHART



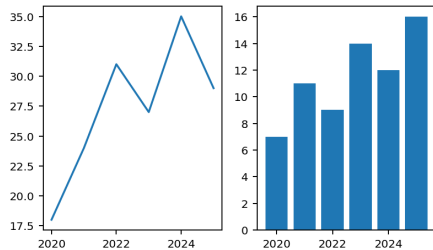
12. Multiple Charts (Subplots)

8 functions

Put several charts on one figure.

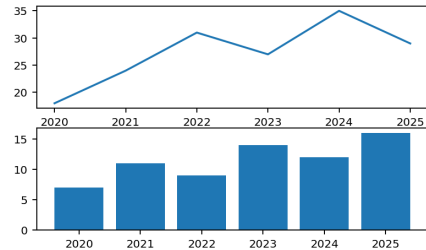
```
12.1 # Two charts side by side
fig, ax = plt.subplots(1, 2)
ax[0].plot(years, hires)
ax[1].bar(years, leavers)
```

CHART



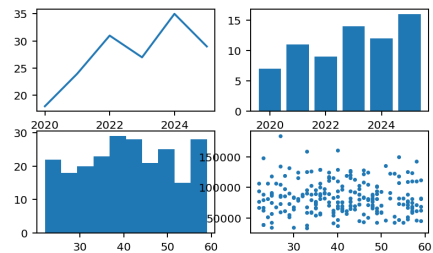
```
12.2 # Two charts stacked vertically
fig, ax = plt.subplots(2, 1)
ax[0].plot(years, hires)
ax[1].bar(years, leavers)
```

CHART



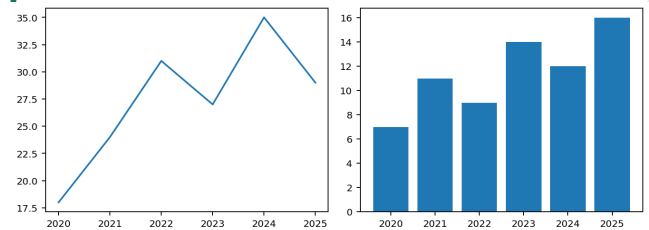
```
12.3 # A grid of four charts
fig, ax = plt.subplots(2, 2)
ax[0, 0].plot(years, hires)
ax[0, 1].bar(years, leavers)
ax[1, 0].hist(df["Age"])
ax[1, 1].scatter(df["Age"], df["Salary"], s=4)
```

CHART



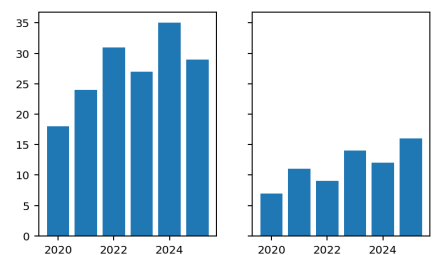
```
12.4 # Stops the charts overlapping
fig, ax = plt.subplots(1, 2, figsize=(8, 3))
ax[0].plot(years, hires)
ax[1].bar(years, leavers)
plt.tight_layout()
```

CHART



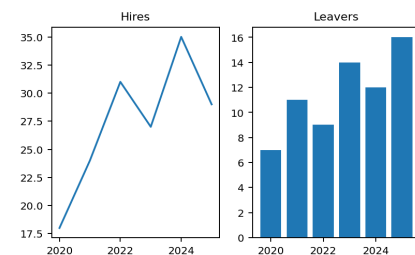
```
12.5 # Both charts share the same scale
fig, ax = plt.subplots(1, 2, sharey=True)
ax[0].bar(years, hires)
ax[1].bar(years, leavers)
```

CHART



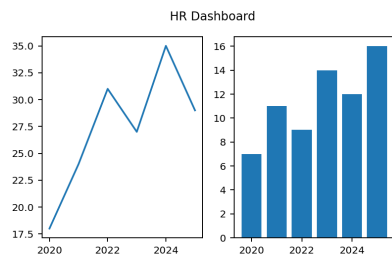
```
12.6 # Gives each chart its own title
fig, ax = plt.subplots(1, 2)
ax[0].plot(years, hires)
ax[0].set_title("Hires")
ax[1].bar(years, leavers)
ax[1].set_title("Leavers")
```

CHART



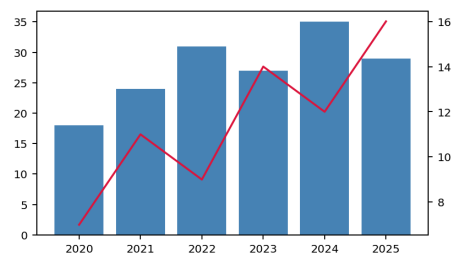
12.7 # One heading above all charts
 fig, ax = plt.subplots(1, 2)
 fig.suptitle("HR Dashboard")
 ax[0].plot(years, hires)
 ax[1].bar(years, leavers)

CHART



12.8 # Two different scales on one chart
 fig, ax = plt.subplots()
 ax2 = ax.twinx()
 ax.bar(years, hires, color="steelblue")
 ax2.plot(years, leavers, color="crimson")

CHART



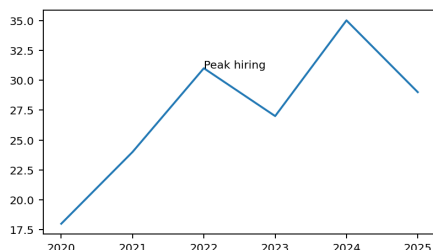
13. Annotations & Text

8 functions

Point out the important bits.

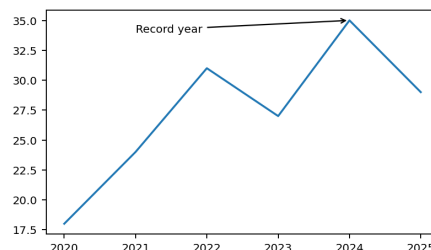
```
13.1 # Writes text at a position
plt.plot(years, hires)
plt.text(2022, 31, "Peak hiring")
```

CHART



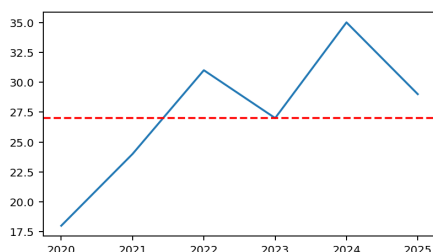
```
13.2 # Adds a labelled arrow
plt.plot(years, hires)
plt.annotate("Record year", xy=(2024, 35), xytext=(2021, 34),
            arrowprops={"arrowstyle": "->"})
```

CHART



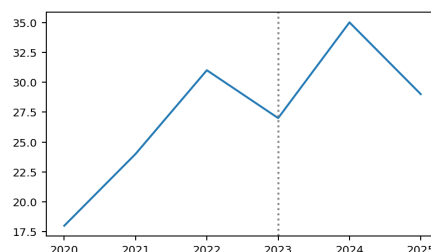
```
13.3 # Draws a horizontal reference line
plt.plot(years, hires)
plt.axhline(y=27, color="red", linestyle="--")
```

CHART



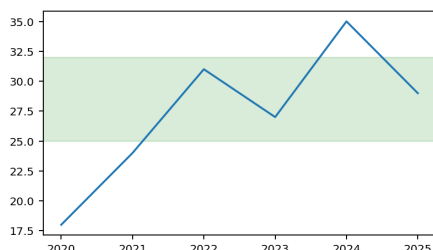
```
13.4 # Draws a vertical reference line
plt.plot(years, hires)
plt.axvline(x=2023, color="grey", linestyle=":")
```

CHART



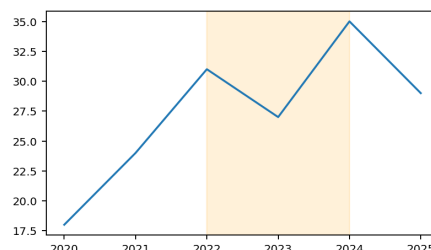
```
13.5 # Shades a target band
plt.plot(years, hires)
plt.axspan(25, 32, color="green", alpha=0.15)
```

CHART



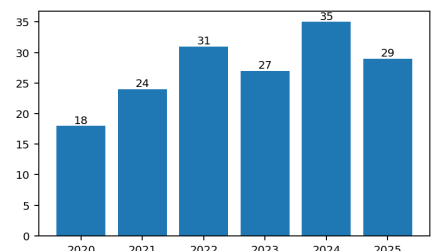
```
13.6 # Highlights a period of time
plt.plot(years, hires)
plt.axvspan(2022, 2024, color="orange", alpha=0.15)
```

CHART



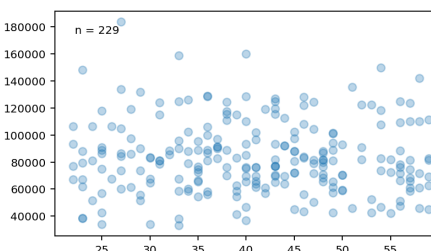
```
13.7 # Labels every bar with its value
bars = plt.bar(years, hires)
plt.bar_label(bars, fmt="%d")
```

CHART



```
13.8 # Places text using chart proportions
plt.scatter(df["Age"], df["Salary"], alpha=0.3)
plt.text(0.05, 0.9, f"n = {len(df)}", transform=plt.gca().transAxes)
```

CHART



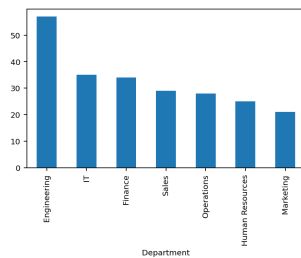
14. Plotting Straight from Pandas

9 functions

The fastest way to chart a DataFrame.

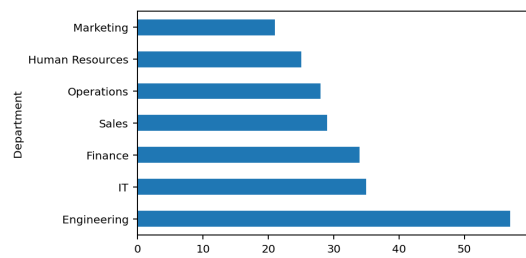
14.1 # Charts a Series directly
dept_counts.plot(kind="bar")

CHART



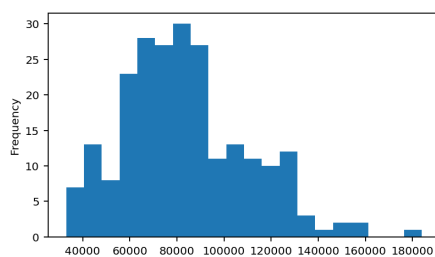
14.2 # Horizontal bars from a Series
dept_counts.plot(kind="barh")

CHART



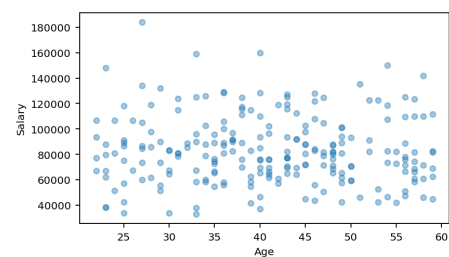
14.3 # Histogram from a column
df["Salary"].plot(kind="hist", bins=20)

CHART



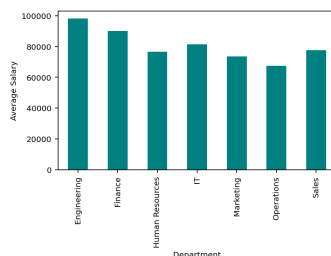
14.4 # Scatter plot from two columns
df.plot(kind="scatter", x="Age", y="Salary", alpha=0.4)

CHART



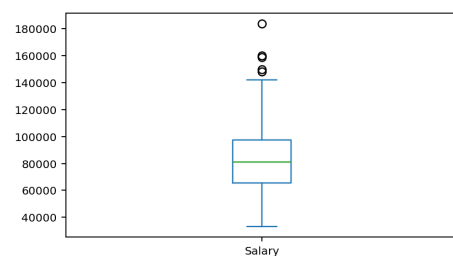
14.5 # Charts a grouped summary
dept_pay.plot(kind="bar", color="teal")
plt.ylabel("Average Salary")

CHART



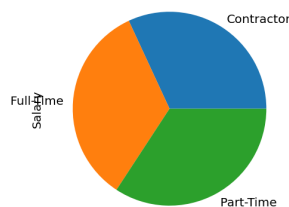
14.6 # Box plot from a column
df["Salary"].plot(kind="box")

CHART



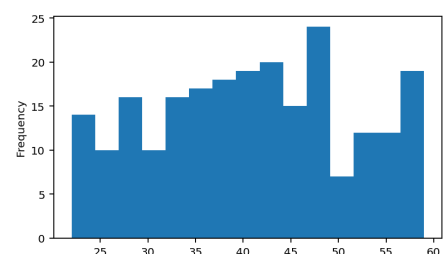
14.7 # Pie chart from a group summary
df.groupby("Employment_Type")["Salary"].mean().plot(kind="pie")

CHART



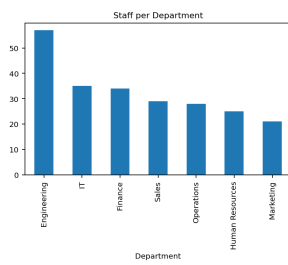
14.8 # Chart one column of a DataFrame
df.plot(kind="hist", y="Age", bins=15, legend=False)

CHART



14.9 # Adds a title while plotting
`dept_counts.plot(kind="bar", title="Staff per Department")`

CHART



15. Saving & Exporting

8 functions

Get the chart out of Python.

15.1 `plt.savefig("chart.png")` # Saves the chart as a PNG file

SAMPLE OUTPUT

1

15.2 `plt.savefig("chart.png", dpi=300)` # High resolution for printing

SAMPLE OUTPUT

58,744 bytes

15.3 `plt.savefig("chart.png", bbox_inches="tight")` # Trims the white space around it

SAMPLE OUTPUT

saved with no wasted margin

15.4 `plt.savefig("chart.pdf")` # Saves as a vector PDF

SAMPLE OUTPUT

1

15.5 `plt.savefig("chart.svg")` # Saves as SVG for the web

SAMPLE OUTPUT

1

15.6 `plt.savefig("chart.png", transparent=True)` # Saves with no background colour

SAMPLE OUTPUT

transparent background saved

15.7 `fig.savefig("chart.png")` # Saves one figure when you have several

SAMPLE OUTPUT

1

15.8 `plt.close("all")` # Closes every open figure

SAMPLE OUTPUT

0